



ARENA FPS

多人第一人称竞技场 Demo

UE5 网络同步与游戏系统技术说明

引擎	Unreal Engine 5.7
网络模式	本地 Dedicated Server / 服务器权威架构
项目仓库	github.com/throusea/arena-fps
文档日期	2026-06-21

01 项目概述与玩法

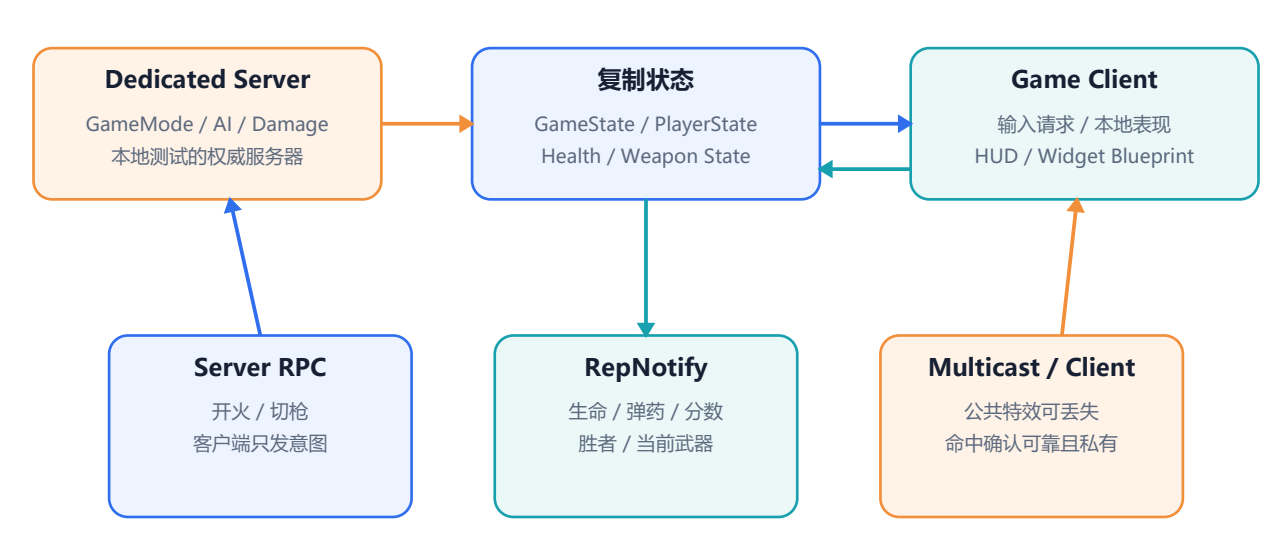
Arena FPS 基于 UE5 官方 First Person 模板开发，是一款多人第一人称竞技场 Demo。项目仅在本地使用 Dedicated Server 进行多人测试，整体采用服务器权威架构。玩家在灰盒竞技场中移动、瞄准、射击、拾取和切换武器，同时对抗其他玩家与服务器控制的敌人。

玩法要素	当前实现
战斗	步枪、手枪、榴弹发射器；Hitscan 与复制投射物；命中、散布、后坐力、装填和特效反馈
敌人	服务器 AI Perception + Behavior Tree；追击最近存活玩家，近身攻击，可受击、死亡和按波次生成
计分	击败敌人 +1 分，击败其他玩家 +3 分，自杀不计分；默认 10 分获胜
重生	玩家死亡后进入死亡镜头，5 秒后在随机 PlayerStart 重生；产生胜者后停止重生
多人 HUD	生命、弹药、装填、武器名、准星、命中确认、个人分数、实时榜单和胜利提示

课程要求覆盖：敌人能够移动和攻击，玩家能够击败敌人；实现得分与胜利；支持多人网络对战。

02 基于网络同步的游戏架构

项目采用 Client-Server 服务器权威模型，并仅在本地通过 Dedicated Server 与多个 Client 进程进行测试。Dedicated Server 不承载本地玩家，只负责权威规则和状态；Client 只提交操作意图，不直接决定伤害、分数、死亡、重生和胜负。



03 网络同步思想

- Reliable Server RPC：开火、停止开火、切换武器等关键输入只提交意图，服务器验证角色状态后执行。
- 属性复制 + OnRep：生命、当前武器、弹药、装填、散布、分数和胜者属于持久状态，保证迟加入客户端也能恢复当前值。
- Unreliable NetMulticast：枪口火焰、普通命中特效和攻击表现频率较高，偶尔丢失不会改变游戏结果。
- Reliable owner-only Client RPC：服务器确认实际造成伤害后，只向射击者返回命中确认，避免客户端猜测与无关广播。
- Authority 边界：AI、Trace、Projectile 生成、伤害、敌人生成、计分、胜利和重生全部由服务器执行。

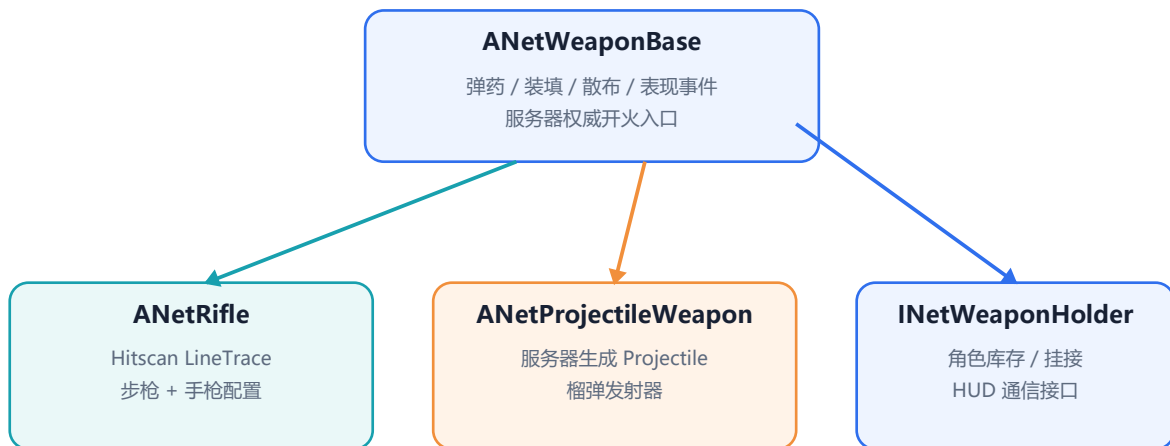
类	网络职责
ANetGameMode	仅服务器存在；差异化计分、胜利检查、重生调度与 PlayerStart 选择
ANetGameState	复制目标分数、胜者；从 PlayerArray 生成全员排序榜单
ANetPlayerStateBase	复制玩家名称与 KillScore，跨 Pawn 重生保留比赛成绩
ANetCharacter / ANetNPC	复制移动、生命、死亡与当前武器；将权威结果路由给表现层
ANetWeaponBase	服务器开火与伤害；复制弹药、装填、散布；分发公共特效和私有命中确认
UNetHUDWidget	仅本地创建；订阅复制数据源的动态委托，调用 Blueprint 表现事件

04 敌人与战斗流程

服务器上的 ANetAIController 使用 AI Perception 维护可见玩家，选择最近的存活目标写入 Blackboard，Behavior Tree 负责追击和攻击决策。ANetNPC::TryAttack 仍会检查权限、距离、冷却和攻击前摇，最终由服务器调用 ApplyDamage。

伤害统一进入 UNetHealthComponent。服务器扣减 Health 并复制 Health 与 bIsDead；NPC 死亡后停止行为树和移动、通知 GameMode 计分并延迟销毁。敌人波次和 SpawnPoint 同样只由服务器驱动，客户端接收已复制的 NPC Actor 与移动结果。

05 武器系统设计



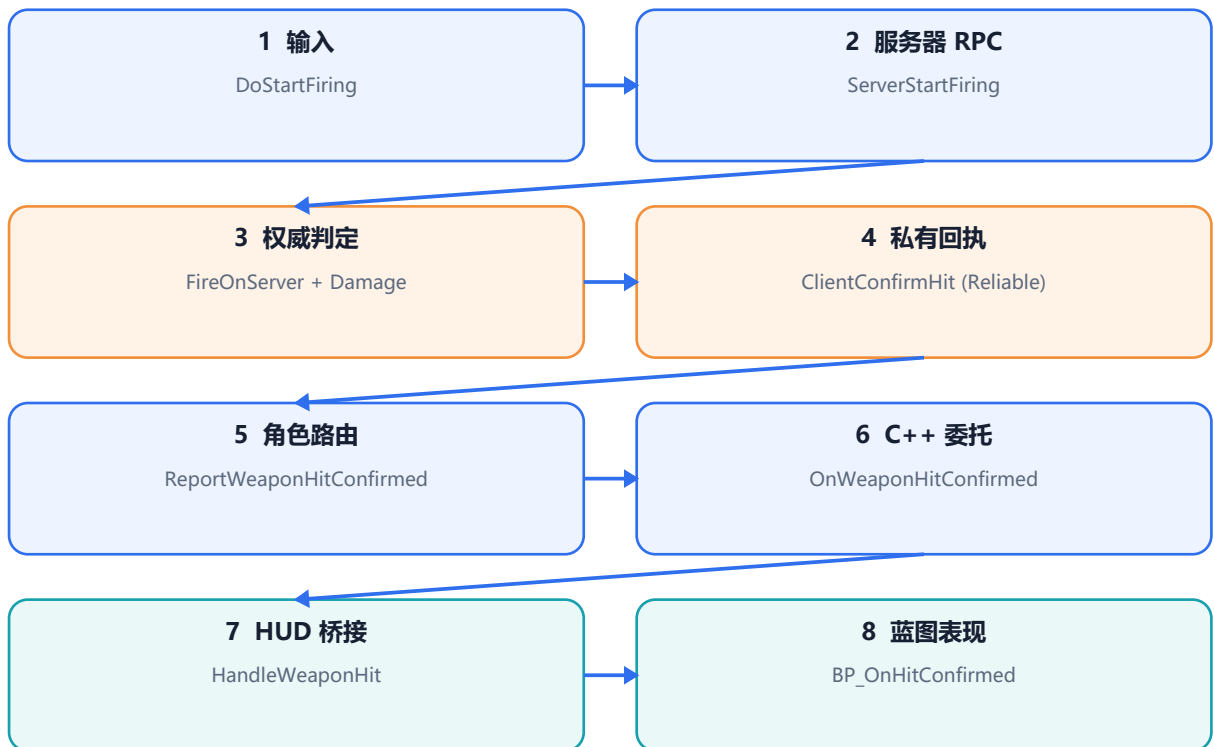
ANetWeaponBase 集中实现弹药、自动装填、射速、连续射击散布、移动/滞空惩罚、散布恢复、第一/第三人称 Mesh、动画、后坐力、Camera Shake、音效、Niagara 以及 C++/Blueprint 表现事件。武器 Actor 启用复制并使用 Owner Relevancy。

- Hitscan 分支：ANetRifle 在服务器从玩家视点计算带散布方向，使用 WeaponTrace 执行 LineTrace 并调用 ApplyPointDamage。步枪和手枪通过 Blueprint 参数形成不同手感。
- Projectile 分支：ANetProjectileWeapon 在服务器生成可复制移动的 ANetProjectile；碰撞后执行单体或范围伤害，榴弹发射器使用这一流程。
- 持有者接口：INetWeaponHolder 解耦武器与角色，负责 Mesh 挂接、动画、后坐力、HUD 更新与命中回报。角色维护武器库存并复制当前武器。
- 拾取系统：ANetWeaponPickup 用 DataTable 配置 Mesh 与武器类，服务器处理重叠授予和定时刷新，防止多个客户端同时领取。

C++ 与 Blueprint 分工：C++ 保证网络规则和数据真实性；Blueprint 配置具体武器参数、资产、动画、特效与 Widget 视觉，兼顾可靠性和快速迭代。

06 BP_OnHitConfirmed 通信设计链路

命中反馈不采用“客户端准星碰到目标就显示”的猜测逻辑，而是等待服务器确认实际伤害。完整事件链如下。



服务器在 `FNetWeaponImpactResult::bDamagedActor == true` 时通过 `ClientConfirmHit` 回执。Owner Client 上的武器调用 `Character` 的 `ReportWeaponHitConfirmed`，`Character` 广播 `OnWeaponHitConfirmed`；`UNetHUDWidget` 订阅后调用 `BP_OnHitConfirmed`，由 `WBP_NetHUD` 播放准星变色或 Hit Marker 动画。

这条链路将职责拆成三段：服务器负责真实性，C++ 委托负责数据路由，Widget Blueprint 负责视觉表现。Reliable owner-only 回执既避免误报，也不会把个人命中反馈广播给所有玩家。

07 计分、胜利、重生与多人榜单



死亡玩家: 5 秒 Timer -> 销毁旧 Pawn -> RestartPlayer -> 随机 PlayerStart

服务器判定死亡后，GameMode 根据目标类型发放分数：敌人 1 分，其他玩家 3 分，自杀不计分。分数写入 PlayerState，因此旧 Pawn 销毁和重新生成不会丢失成绩。每次加分后检查默认 10 分目标，胜者写入 GameState 并复制到所有客户端。

GameState 从 UE 的 PlayerArray 读取所有 ANetPlayerStateBase，按分数降序、名称升序生成 FNetScoreboardEntry。玩家加入、离开、改名或分数变化都会触发 OnScoreboardChanged，两端 HUD 实时重建一致的榜单。

玩家死亡后由服务器启动 5 秒 Timer，随后销毁旧 Pawn 并调用 RestartPlayer。GameMode 每次重生都重新执行随机且避让占用的 PlayerStart 选择；产生胜者后不再安排新重生。

本节覆盖死亡重生、差异化计分与多人实时榜单等核心游戏流程。

08 实现索引与交付信息

系统	核心位置
角色 / 规则	Source/Arena/Variant_Network/NetCharacter.*, NetGameMode.*, NetGameState.*, NetPlayerStateBase.*
生命 / AI	Components/NetHealthComponent.*, NetNPC.*, AI/NetAIController.*, Spawning/*
武器	Weapons/NetRifle.*, NetProjectileWeapon.*, NetProjectile.*, NetWeaponPickup.*
HUD	UI/NetHUDWidget.* 与 Content/Variant_Network/Blueprints/Widgets/
关卡 / 蓝图	Content/Variant_Network/

GitHub 仓库: <https://github.com/throusea/arena-fps>

09 Agent（Codex）与 Skill 使用说明

项目开发过程中使用 Codex 作为辅助开发 Agent。Codex 参与提供部分实现思路、拆解技术方案、编写与重构部分代码、分析网络通信链路，并协助生成和排版本技术文档。功能取舍、工程整合与最终运行结果仍由开发者确认。

Skill	用途
read-project-report	扫描工程结构、配置、源码、资产边界和 Git 状态
github	读取课程需求、功能需求及相关验收标准
documentation / pdf	组织技术说明、生成 PDF，并逐页检查字体、图表、链接和可读性
ue-cpp-foundations	规范 UObject 反射宏、UE 容器、委托与对象生命周期
ue-gameplay-abilities	为 GAS、属性、效果与 Gameplay Tag 等扩展方向提供参考
ue-module-build-system	辅助处理 Build.cs、模块依赖、IWYU 与编译链接问题